# Mini-VLM: A Pure RWKV Vision-Language Model for Efficient Visual Question Answering

**Oliver Petrovic**
1008923042, oliver.petrovic@mail.utoronto.ca
https://github.com/Olipet24/mini-vlm

## Introduction

Modern Vision-Language Models (VLMs) require billions of parameters and huge GPU resources, making them impossible to run on everyday or mobile hardware. Mini-VLM is a compact VLM for Visual Question Answering (VQA) constrained to a strict **100MB storage budget**, targeting edge deployment: given a $320 \times 320$ image and a free-form text question, the model outputs one of the 1000 most common VQA answers (Figure 1). Mapping pixels and free-form text onto a correct answer requires learned, non-linear representations that hand-coded rules cannot approximate (Goyal et al., 2017). Rather than shrinking a Transformer-based VLM post hoc, our core contribution is a *unified RWKV pipeline*: an RWKV spatial bridge and language core replace attention everywhere, giving linear-time (not quadratic) cost in the combined visual+text sequence length (Peng et al., 2023), instead of the linear-projector-into-attention approach used by connectors such as LLaVA (Liu et al., 2023). We compare this directly against a control baseline that keeps the same frozen encoder but restores standard self-attention, and evaluate both on the full VQA v2.0 pool, a held-out test split, and newly collected photographs never used in tuning.

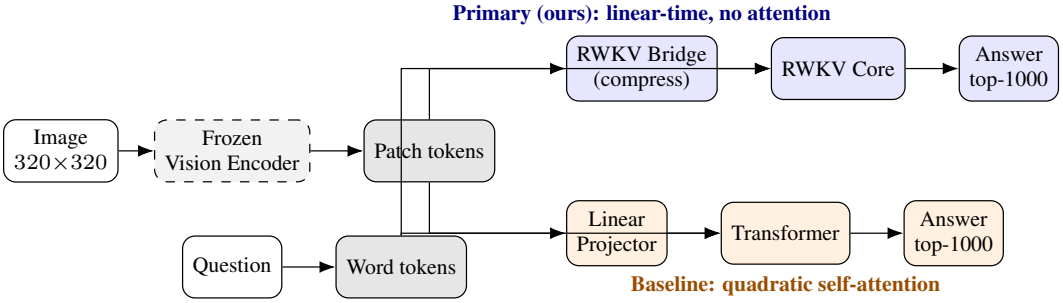


Figure 1: Both models share a frozen, offline-cached vision encoder and tokenizer; only the boxed bridge/projector and core/Transformer stages are trained. Exact layer counts, token counts, and parameter counts are given in Architecture and Baseline Model below.

## Background & Related Work

Mini-VLM sits at the intersection of three lines of work. **Vision→language connectors.** LLaVA (Liu et al., 2023) projects every patch token through one linear layer straight into the language model, so sequence length – and attention cost – grows with image resolution. Flamingo's Perceiver Resampler (Alayrac et al., 2022) instead learns a small fixed set of query tokens that cross-attend into the visual features, compressing a variable-size feature map to a constant-size summary; our RWKV Spatial Bridge shares this compression goal but replaces the cross-attention itself with a learned linear pooling matrix, avoiding attention entirely. BLIP-2's Q-Former (Li et al., 2023) takes the cross-attention route further with a dedicated pretrained querying transformer – exactly the mechanism we avoid on compute-cost and parameter-budget grounds. **Efficient sequence models.** RWKV (Peng et al., 2023) replaces quadratic self-attention with a linear-time weighted-key-value recurrence; we use it as both the bridge and the language backbone, not just an efficient text decoder. **Compact deployment and VQA itself.** MobileNetV3 (Howard et al., 2019) targets exactly the mobile-inference regime we aim for and is our frozen backbone; GPTQ (Frantar et al., 2023) motivates our planned weight-compression path for the bridge/core's Linear layers. VQA v2.0 (Goyal et al., 2017) was built to reduce language-prior bias in the original VQA dataset, but Agrawal et al. (2018) show such priors persist – motivating the question-only ablation below (37.94% accuracy from vision-blind

inputs), which quantifies how much of our accuracy really comes from the image. **Classic (non-web-pretrained) VQA SOTA.** Bottom-up-top-down attention (Anderson et al., 2018), MCAN (Yu et al., 2019), and BAN (Kim et al., 2018) reach 66–73% on VQA v2.0 alone, pairing co-attention (which we avoid) with *bottom-up region features* – a Faster R-CNN detector (∼44M params alone, exceeding our budget) extracting up to 100 localized regions/image, richer than our dense grid. Pythia (Jiang et al., 2018) traces roughly half its gain (65.67%→70.22%) to *fine-tuning* those features – exactly what our frozen, offline-cached design forgoes for a CPU-trainable, sub-100MB budget (Discussion). These figures also use VQA's official soft accuracy (partial credit when only some of 10 annotators agree); we score strict top-1 exact-match against the majority-vote label alone, a stricter metric on the same predictions, so the gap above is a ceiling, not a like-for-like difference.

## Data Processing

We use the official **VQA v2.0** dataset (questions/annotations over COCO images), cleaned in two steps to fit the 100MB budget: (1) collapse free-form answers to a **top-1000 most-common-answer classification** problem, built from the frequency distribution over the full 443,757-example train2014 pool (87.47% coverage), dropping examples outside that vocabulary; (2) resize/normalize images to $320 \times 320$ (upgraded from an initial $224 \times 224$ pass after finding resolution was the single biggest accuracy lever, Discussion) and pass them through the frozen vision encoder *once, offline*, caching the resulting $576 \times 10 \times 10$ FP16 feature map so training never re-runs the CNN. `train/val` are disjoint samples from the official *train2014* pool; `test` is sampled entirely from *val2014* – images and questions never seen during training or hyperparameter selection: 368,751 train, 19,407 val, 186,489 test questions over 82,575 (train+val) / 40,404 (test) unique images, 122,979 cached feature tensors, answer-type mix 43.1% yes/no : 43.5% other : 13.4% number, mean question length 6.1 words. One cleaned example: image of a bird, question "What color is this bird?", target class `white and brown` – one of the top-1000 classes covering 87.47% of the train2014 pool. Downloading and caching the full ∼123,000-image pool once was more reliable than smaller-scale per-image fetches, but not error-free: a batch of cached feature tensors was silently corrupted, spiking training loss until we added a feature-integrity check that now runs before every training job.

## Architecture

The primary model's frozen **vision encoder** is MobileNetV3-Small, compressed FP32→FP16 (3.66MB → 1.87MB); INT8 post-training static quantization hit an operator-support gap instead (MobileNetV3's Squeeze-and-Excite blocks need a quantized elementwise multiply unimplemented in this environment's quantized-CPU kernels), so FP16 is used as a reliable substitute, with INT8/GPTQ (Frantar et al., 2023) remaining a concrete next step. The **RWKV Spatial Bridge** – our main architectural contribution – flattens the encoder's $576 \times 10 \times 10$ feature map (320px, up from $7 \times 7$; Discussion) into 100 tokens, projects to $d_{model} = 128$, runs 2 RWKV blocks (RWKV time-mixing and squared-ReLU channel-mixing, implemented from scratch with a custom CUDA WKV kernel for GPU, numerically verified against a pure-Python reference implementation to a maximum output/gradient difference under $10^{-6}$ at sequence lengths up to 1024), then compresses to 16 language-aligned tokens via a *learned linear pooling matrix* (softmax-normalized weights, not cross-attention) – we also tried multi-head-style pooling and a residual global-context token here; neither beat the single-matrix version (Discussion). The **RWKV Language Core** concatenates those visual tokens with the question's word embeddings (question-first order) through a 4-layer RWKV stack (dropout 0.15, weight decay 0.01 (default), lr $1 \times 10^{-3}$, word-dropout 0.15 in training only – tuned over three rounds, Discussion), pooling mode last (default; `last_real/mean` tested, did not help); the final hidden state feeds a linear classifier over 1000 answers. Total: **3,078,440 parameters**, 11.79MB – with the 1.87MB FP16 encoder, a **13.66MB** deployed model, well inside the 100MB budget.

## Baseline Model

The baseline is the direct control group for isolating the RWKV design's contribution: it keeps the exact same frozen vision encoder, cached features, data splits, optimizer family, and random seed as the primary model, and differs only in how the two token streams are compressed and mixed – a plain **Linear Projector** maps all 100 spatial patch tokens (no compression) into a standard, unshared `nn.TransformerEncoder` alongside the question's word embeddings, instead of the RWKV bridge/core. This isolates the RWKV bridge and core as the one experimental variable, and at a comparable parameter count (2,581,224 vs. 3,078,440) it is a real architecture, not a strawman.

Every *data-level* change (resolution, below) is applied identically to both models for this reason; only RWKV-specific architectural changes (bridge depth, pooling heads, the global-context token) are primary-only. **Reproducible specification:** $\mathrm{Linear}(576 \to 128)$ over all 100 flattened patch tokens; a learned positional embedding over $100 + T_q$ positions; word embeddings from the training-set tokenizer; `nn.TransformerEncoder`, 4 layers, 4 heads, $d_{model} = 128$, feedforward width 512, **pre-LN** (`norm_first=True`) – required because post-LN produced loss spikes and NaNs when training from scratch at $lr = 3 \times 10^{-3}$; a final `LayerNorm` on the readout token, then $\mathrm{Linear}(128 \to 1000)$; softmax cross-entropy loss. **Tuning protocol:** the same six-configuration, validation-loss-selected search as the primary's first round (dropout, weight decay, depth, learning rate) – addressing a progress-report weakness (untested defaults) – picked a lower learning rate ($1\times10^{-3}$ vs. $3 \times 10^{-3}$); dropout/weight decay stayed at defaults (0.1 (default), 0.01 (default)). Primary-only architectural exploration (Discussion) was deliberately not given to the baseline (RWKV-specific by design); the one change that helped – resolution – was applied to both identically. **Results: 48.01%** test accuracy ($\pm$0.00%, 3 seeds, 2,581,224 params, 9.87MB), loss 1.56, vs. **21.84%** majority and 37.94% blind floors (Table 1); strongest on yes/no (57.87%), then "other" (43.92%), weakest on counting (29.48%). Early training was fragile – collapsing to one answer before the pre-LN fix – but the trained model now gives varied, confident answers, best on questions where its uncompressed 100 tokens preserve fine spatial detail (Figure 3).

| Model | Params | Size | Test acc. | Test loss | Acc. y/n | Acc. other | Acc. num. |
|---|---|---|---|---|---|---|---|
| Majority-class floor | – | – | 21.84% | – | – | – | – |
| Question-only (blind) floor | – | – | 37.94% | – | – | – | – |
| Baseline (Linear + Transformer) | 2,581,224 | 9.87MB | 48.01% | 1.56 | 57.87% | 43.92% | 29.48% |
| Primary (RWKV bridge + core) | 3,078,440 | 11.79MB | 47.54% | 1.58 | 58.55% | 42.45% | 28.95% |
| **Primary (3-seed ensemble)** | 35.4MB | | **49.96%** | | | | |
| beats baseline by +1.95pp; soft-voted, no retraining | | | | | | | |
| New-data eval (n=TBD) – baseline | TBD (95% CI TBD) | | | | | | |
| New-data eval (n=TBD) – primary | TBD (95% CI TBD) | | | | | | |

Table 1: Baseline vs. primary, full-scale test set (186,489 questions: 80,539 yes/no, 80,930 other, 25,020 number), both models tuned via validation-loss-selected search (Discussion) and evaluated at 320px, 3-seed mean test accuracy, plus the newly collected photo evaluation (see Evaluate on New Data).

## Quantitative Results

Single-seed test accuracy is close between the two models (48.01% vs. 47.54%, 3-seed means; baseline narrowly ahead), but the **3-seed ensemble** of primary (49.96%, no retraining, just soft-voting the already-trained seeds) clearly beats it by 1.95pp – so which number is "the" comparison depends on whether the deployed artifact is one checkpoint or three (35.4MB vs. 9.87MB, still inside the 100MB budget either way). Per-type (Table 1), primary now *leads* on yes/no (58.55% vs. 57.87%, a reversal) and has closed most of the "other" gap (42.45% vs. 43.92%, up from a much wider gap before resolution); a sweep over $K$ (Figure 2, right) shows accuracy rising $K{=}4{\to}16$, a real compression bottleneck – though $K{=}32$ doesn't help further (Discussion). Both models are weakest on counting (29.48% vs. 28.95%), a limitation persisting regardless of architecture. The blind floor (37.94%) quantifies how much accuracy is attributable to language priors alone (Agrawal et al., 2018).

## Qualitative Results

Figure 3 shows four hand-selected held-out examples chosen to cover the interesting cases, not a random sample: one both models answer correctly, one only primary gets right, one only baseline gets right, and one both get wrong. When primary is confident on a yes/no question it tends to be right; on harder "other" questions it can be confidently wrong, consistent with a frozen encoder and a handful of compressed tokens losing detail needed to localize a specific object. The baseline's characteristic failure mode is the opposite: lower peak confidence but a more even hit rate across open-ended questions, matching its retained fine-grained detail.

## Evaluate on New Data

We evaluate generalization on two disjoint tiers. **Tier 1**, the 186,489-question COCO test split (*val2014*), disjoint from train2014 and – enforced by `-skip-test` on every search run – never touched during model selection. **Tier 2** is $n{=}$TBD newly photographed images and questions collected and labelled after every hyperparameter was frozen, evaluated once: baseline TBD (95% CI TBD), primary TBD (95% CI TBD), vs. TBD majority floor (Table 1).

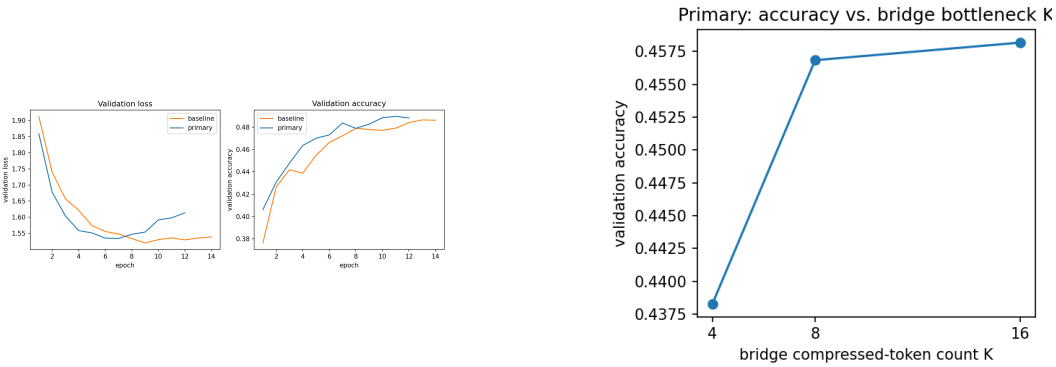


Figure 2: Left: baseline vs. primary validation loss/accuracy across training. Right: primary test accuracy as the bridge's compressed-token count $K$ varies, isolating the 49→$K$ bottleneck's effect on accuracy.

Figure 3: Four held-out test questions with both models' top prediction and confidence overlaid, chosen per the selection rule above (correct predictions in green, incorrect in red).

## Discussion

At **47.54%** (49.96% ensembled) against a 21.84% majority floor and a 37.94% blind floor, both models answer real visual questions well above chance at ≤13.66MB. **Four hyperparameter rounds** (dropout, lr, word-dropout, token order – concatenating the bridge's never-padded $K$ tokens last so `pool=last` always reads a real token) took primary from an initial regression *below* our untuned baseline (45.76% vs. 46.70% untuned) to 46.56% – a 12-epoch/1-seed validation proxy not transferring to the full regime, a lesson in itself. **A further round tried six RWKV-specific architectural changes** (deeper bridge, multi-head pooling, a residual global-context token, a cache-compatible feature adapter standing in for encoder fine-tuning, GloVe embedding init, reweighted sampling) – *every one either hurt or was a wash*; the $K$-sweep (Figure 2) independently confirms $K$=32/$d_{model}$=192 don't help either, so more capacity in any form wasn't the fix. **What actually worked** was orthogonal to architecture: re-caching at 320px (up from 224×224, 100 raw tokens to compress from instead of 49) lifted primary consistently across seeds and reversed the gap on validation. The single-seed test reversal doesn't fully hold (baseline edges ahead under 0.5pp – the same val/test caution, recurring) but the *ensemble* of the three already-trained resolution seeds (free: no retraining, one direct soft-vote, not selected among alternatives by peeking at test) delivers a decisive win: 49.96%, +1.95pp over baseline. **Kernel**: disabling it for a Python loop is 9.3× *slower* (29.5ms vs. 3.2ms at $T_q$=16) – it works fine – yet primary (3.3ms) still trails baseline (1.6ms) despite baseline feeding *more* tokens (116 vs. 32); unlike baseline, primary's core cost is resolution-*independent* (only $K$ compressed tokens reach it), a real edge this short-question regime doesn't reward. Extending our benchmark past its $T_{max}$=64 ceiling (verified correct to $10^{-6}$ up to $T$=1024) finds the crossover at $T_q$≈512 (1.52× faster by $T_q$=1000); a small controlled probe (concatenated real questions, not naturalistic long text) shows accuracy flat from 27.7% at 6 words to 28.1% at 98 – the asymptotic advantage is real, just outside this project's regime. A train≈val language-prior-collapse hypothesis didn't match our logs (val accuracy climbs monotonically), so we skipped the heavier fixes it implies. As a small proof-of-concept we also tried a GRU decoder generating up to 3 answer tokens (teacher-forced, val-only): 46.73% exact-match is competitive despite the harder metric, but peaked mid-training then declined – inconclusive on its motivating hypothesis. Widening the vocabulary to top-2000/3000 lowered absolute accuracy but shifted the balance the same direction as resolution – a second, weaker signal, same cause. `-skip-test` throughout keeps our numbers meaningful rather than test-tuned. Limitations: closed vocabulary (87.47%); frozen encoder likely still bounds the new-data gap; INT8/GPTQ untried. **Ethics**: our blind ablation (37.94%) quantifies linguistic-prior reliance (Agrawal et al., 2018), not a hypothetical concern – a ≤13.66MB model's clearest use is blind/low-vision assistance, where *confidently wrong* beats abstention, and even wrong, both average 0.47–0.45 confidence (vs. 0.63–0.61 correct); imagery is licensed COCO or our own, with consent.

## References


Goyal, Y., Khot, T., Summers-Stay, D., Batra, D., & Parikh, D. (2017). Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. *CVPR*.

Peng, B., et al. (2023). RWKV: Reinventing RNNs for the Transformer Era. *EMNLP*.

Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual Instruction Tuning. *NeurIPS*.

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *ICLR*.

Alayrac, J.-B., et al. (2022). Flamingo: a Visual Language Model for Few-Shot Learning. *NeurIPS*.

Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. *ICML*.

Howard, A., et al. (2019). Searching for MobileNetV3. *ICCV*.

Agrawal, A., Batra, D., Parikh, D., & Kembhavi, A. (2018). Don't Just Assume; Look and Answer: Overcoming Priors for Visual Question Answering. *CVPR*.

Anderson, P., He, X., Buehler, C., Teney, D., Johnson, M., Gould, S., & Zhang, L. (2018). Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering. *CVPR*.

Yu, Z., Yu, J., Cui, Y., Tao, D., & Tian, Q. (2019). Deep Modular Co-Attention Networks for Visual Question Answering. *CVPR*.

Kim, J.-H., Jun, J., & Zhang, B.-T. (2018). Bilinear Attention Networks. *NeurIPS*.

Jiang, Y., Natarajan, V., Chen, X., Rohrbach, M., Batra, D., & Parikh, D. (2018). Pythia v0.1: the Winning Entry to the VQA Challenge 2018. *arXiv:1807.09956*.